

CURSO 4: TALLER PRÁCTICO & PROYECTO FINAL INTEGRADOR

CLASE 02 • NIVEL 4 - INTEGRADOR

Clase 02: Desarrollo del Backend: APIs RESTful con FastAPI

«FastAPI como un Centro Logístico de Alta Velocidad para Peticiones HTTP»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)
Rol: AI Solutions Architect & Principal Software Engineer
Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 02** del **Curso 4: Taller Práctico & Proyecto Final Integrador**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender el protocolo HTTP (GET, POST, PUT, DELETE), códigos de estado (200, 201, 404, 500) y asincronía (async/await).

Competencia Práctica: Construir una API REST completa con documentación interactiva Swagger automática.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 02: Desarrollo del Backend: APIs RESTful con FastAPI

FastAPI es el framework web moderno de Python más rápido, diseñado para construir microservicios y APIs con tipado estricto.

☀ Metáfora Central: FastAPI como un Centro Logístico de Alta Velocidad para Peticiones HTTP

FastAPI es una ventanilla de atención ultra rápida: valida tu formulario antes de atenderte y te entrega un recibo oficial.

Principios Teóricos y Modelo Mental

Validación automática de requests y responses gracias a la integración profunda con Pydantic.

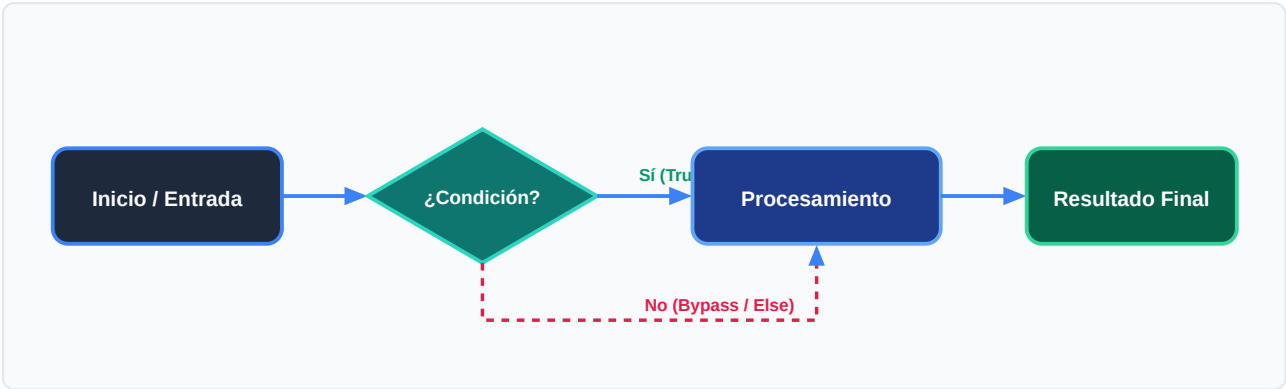
Generación automática de documentación interactiva en /docs (Swagger UI) y /redoc.

⚡ Regla de Oro en Python

Retorna siempre códigos de estado HTTP semánticos (ej. 201 Created tras un POST exitoso).

Arquitectura y Movimiento de Datos en Memoria

Flujo de una petición HTTP en FastAPI desde el router hasta la respuesta JSON.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Cliente envía HTTP Request (JSON body + headers).	Petición recibida en puerto 8000.
2. Evaluación	Validación automática del esquema Pydantic.	DTO validado o 422 Unprocessable Entity.
3. Transformación	Ejecución de la función de ruta (Endpoint).	Lógica de negocio ejecutada.
4. Retorno / Salida	Serialización del resultado y retorno de HTTP Response.	JSON emitido con código 200/201.

Visualización Mental

Cada función decorada con `@app.get` o `@app.post` representa un punto de entrada para clientes.

Código de Demostración en Python 3.10+

API REST funcional con FastAPI y modelos Pydantic:

main.py (Python 3.10+)

PEP 8 Compliant

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Servicio de Productos API", version="1.0.0")

class Producto(BaseModel):
    id: int
    nombre: str
    precio: float

DB_ITEMS = {}

@app.post("/productos", status_code=201)
def crear_producto(prod: Producto):
    if prod.id in DB_ITEMS:
        raise HTTPException(status_code=400, detail="El producto ya existe.")
    DB_ITEMS[prod.id] = prod
    return {"mensaje": "Creado con éxito", "producto": prod}

@app.get("/productos/{item_id}")
def obtener_producto(item_id: int):
    if item_id not in DB_ITEMS:
        raise HTTPException(status_code=404, detail="No encontrado")
    return DB_ITEMS[item_id]
```

Análisis de Ingeniería del Código

Manejo explícito de códigos 201 y 404, validación automática y documentación interactiva.

Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Errores de concurrencia y bloqueos en async.

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar funciones síncronas bloqueantes (como `time.sleep`) dentro de funciones `async def` congela todo el servidor.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
async def endpoint():
    time.sleep(5) # ❌ Bloquea el event loop
    para todos los usuarios
```

✅ Patrón Correcto

```
async def endpoint():
    await asyncio.sleep(5) # ✅ No bloqueante
```

🛡 Consejo de Resiliencia en Producción

Utiliza `Depends()` para inyectar conexiones de base de datos y autenticación limpia.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 02: Desarrollo del Backend: APIs RESTful con FastAPI**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	FastAPI como un Centro Logístico de Alta Velocidad para Peticiones HTTP
2. Regla de Oro	Retorna siempre códigos de estado HTTP semánticos (ej. 201 Created tras un POST exitoso).
3. Gotcha Crítico	Usar funciones síncronas bloqueantes (como time.sleep) dentro de funciones async def congela todo el servidor.
4. Buenas Prácticas	Utiliza Depends() para inyectar conexiones de base de datos y autenticación limpia.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Añade endpoints PUT (actualizar) y DELETE a la API de productos con validación de existencia.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.